Task 12

Using only AWS CLI:

Create a custom VPC

Create 2 public subnets (pub/pvt)
Create Internet Gateway
Create NAT Gateway
Create Route Table
Create Security Group
Launch 2 EC2 instances (1 in each Subnet)
Install Nginx via user data
Create Application Load Balancer
Register instances
Test via ALB DNS

```
suhani@debian:~$ aws ec2 create-vpc \
--cidr-block 10.0.0.0/16 \
--tag-specifications '[{"ResourceType":"vpc","Tags":[{"Key":"Name","Value":"MyVpc"}]}]'
{
    "Vpc": {
        "CidrBlock": "10.0.0.0/16",
        "DhcpOptionsId": "dopt-0273ad08a6a727b96",
        "State": "pending",
        "VpcId": "vpc-032a761867508e4ea",
        "OwnerId": "966001267394",
        "InstanceTenancy": "default",
        "Ipv6CidrBlockAssociationSet": [],
        "CidrBlockAssociationSet": [
            {
                "AssociationId": "vpc-cidr-assoc-0ffd38b2d588844e3",
                "CidrBlock": "10.0.0.0/16",
                "CidrBlockState": {
                    "State": "associated"
                }
            }
        ],
        "IsDefault": false,
        "Tags": [
            {
                "Key": "Name",
                "Value": "MyVpc"
            }
:...skipping...
{
    "Vpc": {
        "CidrBlock": "10.0.0.0/16",
        "DhcpOptionsId": "dopt-0273ad08a6a727b96",
        "State": "pending",
        "VpcId": "vpc-032a761867508e4ea",
        "OwnerId": "966001267394",
        "InstanceTenancy": "default",
        "Ipv6CidrBlockAssociationSet": [],
        "CidrBlockAssociationSet": [
            {
                "AssociationId": "vpc-cidr-assoc-0ffd38b2d588844e3",
                "CidrBlock": "10.0.0.0/16",
                "CidrBlockState": {
                    "State": "associated"
                }
            }
        ],
        "IsDefault": false,
        "Tags": [
            {
                "Key": "Name",
                "Value": "MyVpc"
            }
        ]
    }
}
~
```

```
{
    "Vpc": {
        "CidrBlock": "10.0.0.0/16",
        "DhcpOptionsId": "dopt-0273ad08a6a727b96",
        "State": "pending",
        "VpcId": "vpc-032a761867508e4ea",
        "OwnerId": "966001267394",
        "InstanceTenancy": "default",
        "Ipv6CidrBlockAssociationSet": [],
        "CidrBlockAssociationSet": [
            {
                "AssociationId": "vpc-cidr-assoc-0ffd38b2d588844e3",
                "CidrBlock": "10.0.0.0/16",
                "CidrBlockState": {
                    "State": "associated"
                }
            }
        ],
        "IsDefault": false,
        "Tags": [
            {
                "Key": "Name",
                "Value": "MyVpc"
            }
:
```


Step 2: Create Subnets

Two subnets were created:

- **Public Subnet** (for Internet access)
- **Private Subnet** (for secure instances)

```
~ : bash   (admin) 13.60.69.11   ~ : bash   ~ : bash
suhani@debian:~$ aws ec2 create-subnet --vpc-id "vpc-032a761867508e4ea" --cidr-block 10.0.1.0/24 --availability-zone ap-south-1a
{
    "Subnet": {
        "AvailabilityZone": "ap-south-1a",
        "AvailabilityZoneId": "aps1-az1",
        "AvailableIpAddressCount": 251,
        "CidrBlock": "10.0.1.0/24",
        "DefaultForAz": false,
        "MapPublicIpOnLaunch": false,
        "MapCustomerOwnedIpOnLaunch": false,
        "State": "available",
        "SubnetId": "subnet-001b00325d5848dca",
        "VpcId": "vpc-032a761867508e4ea",
        "OwnerId": "966001267394",
        "AssignIpv6AddressOnCreation": false,
        "Ipv6CidrBlockAssociationSet": [],
        "SubnetArn": "arn:aws:ec2:ap-south-1:966001267394:subnet/subnet-001b00325d5848dca"
    }
}
suhani@debian:~$ aws ec2 create-subnet --vpc-id "vpc-032a761867508e4ea" --cidr-block 10.0.2.0/24 --availability-zone ap-south-1b
{
    "Subnet": {
        "AvailabilityZone": "ap-south-1b",
        "AvailabilityZoneId": "aps1-az3",
        "AvailableIpAddressCount": 251,
        "CidrBlock": "10.0.2.0/24",
        "DefaultForAz": false,
        "MapPublicIpOnLaunch": false,
        "MapCustomerOwnedIpOnLaunch": false,
        "State": "available",
        "SubnetId": "subnet-0088364c14b7919b0",
        "VpcId": "vpc-032a761867508e4ea",
        "OwnerId": "966001267394",
        "AssignIpv6AddressOnCreation": false,
        "Ipv6CidrBlockAssociationSet": [],
        "SubnetArn": "arn:aws:ec2:ap-south-1:966001267394:subnet/subnet-0088364c14b7919b0"
    }
}
suhani@debian:~$
```

Public subnet was configured to auto-assign public IPs.

Step 3: Create Internet Gateway

```
suhani@debian:~$ aws ec2 create-internet-gateway
{
    "InternetGateway": {
        "Attachments": [],
        "InternetGatewayId": "igw-0003eff91ae7480de",
        "OwnerId": "966001267394",
        "Tags": []
    }
}
suhani@debian:~$
```

Step 4: Create Route Table for Public Subnet

```
suhani@debian:~$ aws ec2 attach-internet-gateway --vpc-id "vpc-032a761867508e4ea" --internet-gateway-id "igw-0
003eff91ae7480de"
suhani@debian:~$
```

Associated with **public subnet**.

Step 5: Create NAT Gateway

Allocate Elastic IP:

```
suhani@debian:~$ aws ec2 allocate-address --domain vpc
{
    "PublicIp": "13.206.39.190",
    "AllocationId": "eipalloc-089bcb7cbff077739",
    "PublicIpv4Pool": "amazon",
    "NetworkBorderGroup": "ap-south-1",
    "Domain": "vpc"
}
```

Create NAT Gateway (in public subnet):

```
suhani@debian:~$ aws ec2 create-nat-gateway \
--subnet-id "subnet-001b00325d5848dca" \
--allocation-id "eipalloc-089bcb7cbff077739"
{
    "ClientToken": "daefe45d-5052-4db7-ace5-4f708cf8b71c",
    "NatGateway": {
        "CreateTime": "2026-04-30T00:41:53+00:00",
        "NatGatewayAddresses": [
            {
                "AllocationId": "eipalloc-089bcb7cbff077739"
            }
        ],
        "NatGatewayId": "nat-050d3c9606d937860",
        "State": "pending",
        "SubnetId": "subnet-001b00325d5848dca",
        "VpcId": "vpc-032a761867508e4ea"
    }
}
```

Step 6: Configure Private Route Table

```
suhani@debian:~$ aws ec2 attach-internet-gateway --vpc-id "vpc-032a761867508e4ea" --internet-gateway-id "igw-0003eff91ae7480de"
suhani@debian:~$ aws ec2 create-route-table --vpc-id "vpc-032a761867508e4ea"
{
    "RouteTable": {
        "Associations": [],
        "PropagatingVgws": [],
        "RouteTableId": "rtb-08f397dde0de4d4d2",
        "Routes": [
            {
                "DestinationCidrBlock": "10.0.0.0/16",
                "GatewayId": "local",
                "Origin": "CreateRouteTable",
                "State": "active"
            }
        ],
        "Tags": [],
        "VpcId": "vpc-032a761867508e4ea",
        "OwnerId": "966001267394"
    }
}
suhani@debian:~$ aws ec2 create-route \                                    --route-table-id "rtb-08f397dde0de4d4d2"  \
--destination-cidr-block 0.0.0.0/0 \
--gateway-id "igw-0003eff91ae7480de"
{
    "Return": true
}
```

Associated with **private subnet**.

```
}
suhani@debian:~$ aws ec2 create-route-table --vpc-id "vpc-032a761867508e4ea"
{
    "RouteTable": {
        "Associations": [],
        "PropagatingVgws": [],
        "RouteTableId": "rtb-0fda057f2d92c629c",
        "Routes": [
            {
                "DestinationCidrBlock": "10.0.0.0/16",
                "GatewayId": "local",
                "Origin": "CreateRouteTable",
                "State": "active"
            }
        ],
        "Tags": [],
        "VpcId": "vpc-032a761867508e4ea",
        "OwnerId": "966001267394"
    }
}
suhani@debian:~$ aws ec2 create-route \
--route-table-id "rtb-0fda057f2d92c629c" \
--destination-cidr-block 0.0.0.0/0 \
--nat-gateway-id "nat-050d3c9606d937860"
{
    "Return": true
}
```

Step 7: Create Security Group

```
suhani@debian:~$ aws ec2 create-security-group \
--description "Allow HTTP" \
--vpc-id "vpc-032a761867508e4ea"
{
    "GroupId": "sg-0b3fb290e628148ef"
}
```

```
}
suhani@debian:~$ aws ec2 authorize-security-group-ingress \
--group-id "sg-0b3fb290e628148ef" \
--protocol tcp --port 80 --cidr 0.0.0.0/0 \
> --protocol tcp --port 22 --cidr 0.0.0.0/0
```

Step 8: Launch EC2 Instances

- Instance 1 → Public Subnet
- Instance 2 → Private Subnet

User Data:

```
suhani@debian:~$ cat > userdata.sh <<EOF
#!/bin/bash
apt update -y
apt install nginx -y
systemctl start nginx
systemctl enable nginx
echo "Hello from $(hostname)" > /var/www/html/index.html
EOF
```

```
suhani@debian:~$ aws ec2 run-instances \
--image-id ami-0f5ee92e2d63afc18 \
--instance-type t2.micro \
--subnet-id "subnet-001b00325d5848dca" \
--security-group-ids "sg-0b3fb290e628148ef" \
--associate-public-ip-address \
--user-data file://userdata.sh \
--count 1
{
    "Groups": [],
    "Instances": [
        {
            "AmiLaunchIndex": 0,
            "ImageId": "ami-0f5ee92e2d63afc18",
            "InstanceId": "i-0bdb3b16c9ed3b455",
            "InstanceType": "t2.micro",
            "LaunchTime": "2026-04-30T00:57:03+00:00",
            "Monitoring": {
                "State": "disabled"
            },
            "Placement": {
                "AvailabilityZone": "ap-south-1a",
                "GroupName": "",
                "Tenancy": "default"
            },
            "PrivateDnsName": "ip-10-0-1-198.ap-south-1.compute.int
            "PrivateIpAddress": "10.0.1.198",
            "ProductCodes": [],
            "PublicDnsName": "",
            "State": {
                "Code": 0,
                "Name": "pending"
```

```
suhani@debian:~$ aws ec2 run-instances \
--image-id ami-0f5ee92e2d63afc18 \
--instance-type t2.micro \
--subnet-id "subnet-0088364c14b7919b0" \
--security-group-ids "sg-0b3fb290e628148ef" \
--user-data file://userdata.sh \
--count 1
{
    "Groups": [],
{
    "Groups": [],
    "Instances": [
        {
            "AmiLaunchIndex": 0,
            "ImageId": "ami-0f5ee92e2d63afc18",
            "InstanceId": "i-0dfb704863051188a",
            "InstanceType": "t2.micro",
            "LaunchTime": "2026-04-30T00:59:29+00:00",
            "Monitoring": {
                "State": "disabled"
            },
            "Placement": {
                "AvailabilityZone": "ap-south-1b",
```

Step 9: Create Target Group

```
suhani@debian:~$ aws elbv2 create-target-group \
--name my-targets \
--protocol HTTP \
--port 80 \
--vpc-id "vpc-032a761867508e4ea"
{
    "TargetGroups": [
        {
            "TargetGroupArn": "arn:aws:elasticloadbalancing:ap-south-1:966001267394:targetgroup/my-targets/9ab2b86bcd1a70cd"
            "TargetGroupName": "my-targets",
            "Protocol": "HTTP",
            "Port": 80,
            "VpcId": "vpc-032a761867508e4ea",
            "HealthCheckProtocol": "HTTP",
            "HealthCheckPort": "traffic-port",
            "HealthCheckEnabled": true,
            "HealthCheckIntervalSeconds": 30,
            "HealthCheckTimeoutSeconds": 5,
            "HealthyThresholdCount": 5,
            "UnhealthyThresholdCount": 2,
            "HealthCheckPath": "/",
            "Matcher": {
                "HttpCode": "200"
            },
            "TargetType": "instance"
        }
    ]
}
(END)
```

Step 10: Register Instances

```
~ : bash   ~ : bash   ~ : bash   ~ : bash   ~ : bash   ~ : aws   ~ : aws
suhani@debian:~$ aws elbv2 register-targets \
--target-group-arn "arn:aws:elasticloadbalancing:ap-south-1:966001267394:targetgroup/my-targets/9ab2b86bcd1a70cd" \
--targets Id="i-0dfb704863051188a" Id="i-0bdb3b16c9ed3b455"
suhani@debian:~$ aws elbv2 create-load-balancer \
--name my-alb \
--subnets "subnet-001b00325d5848dca" "subnet-0088364c14b7919b0" \
--security-groups "sg-0b3fb290e628148ef"
}
{
    "LoadBalancers": [
        {
            "LoadBalancerArn": "arn:aws:elasticloadbalancing:ap-south-1:966001267394:loadbalancer/app/my-alb/811106bb33a240e
            "DNSName": "my-alb-1576251582.ap-south-1.elb.amazonaws.com",
            "CanonicalHostedZoneId": "ZP97RAFLXTNZK",
            "CreatedTime": "2026-04-30T01:06:51.261000+00:00",
            "LoadBalancerName": "my-alb",
            "Scheme": "internet-facing",
            "VpcId": "vpc-032a761867508e4ea",
            "State": {
                "Code": "provisioning"
            },
            "Type": "application",
            "AvailabilityZones": [
                {
                    "ZoneName": "ap-south-1a",
                    "SubnetId": "subnet-001b00325d5848dca",
                    "LoadBalancerAddresses": []
                },
                {
                    "ZoneName": "ap-south-1b",
                    "SubnetId": "subnet-0088364c14b7919b0",
```

Step 11: Create Application Load Balancer

```
~ : bash   ~ : bash   ~ : bash   ~ : bash   ~ : bash   ~ : aws   ~ : aws
suhani@debian:~$ aws elbv2 create-listener \
--load-balancer-arn "arn:aws:elasticloadbalancing:ap-south-1:966001267394:loadbalancer/app/my-alb/811106bb33a240ed" \
--protocol HTTP \
--port 80 \                                                    --default-actions Type=forward,TargetGroupArn="arn:aws:elasticloadbalancing:ap-south-1:966001267394:targetgroup/my-targets/9ab
2b86bcd1a70cd"
{
    "Listeners": [
        {
            "ListenerArn": "arn:aws:elasticloadbalancing:ap-south-1:966001267394:listener/app/my-alb/811106bb33a240ed/76791ce77f67fe75",
            "LoadBalancerArn": "arn:aws:elasticloadbalancing:ap-south-1:966001267394:loadbalancer/app/my-alb/811106bb33a240ed",
            "Port": 80,
            "Protocol": "HTTP",
            "DefaultActions": [
                {
                    "Type": "forward",
                    "TargetGroupArn": "arn:aws:elasticloadbalancing:ap-south-1:966001267394:targetgroup/my-targets/9ab2b86bcd1a70cd",
                    "ForwardConfig": {
                        "TargetGroups": [
                            {
                                "TargetGroupArn": "arn:aws:elasticloadbalancing:ap-south-1:966001267394:targetgroup/my-targets/9ab2b86bcd1a70cd",
                                "Weight": 1
                            }
                        ],
                        "TargetGroupStickinessConfig": {
                            "Enabled": false
:...skipping...
{
    "Listeners": [
        {
            "ListenerArn": "arn:aws:elasticloadbalancing:ap-south-1:966001267394:listener/app/my-alb/811106bb33a240ed/76791ce77f67fe75",
            "LoadBalancerArn": "arn:aws:elasticloadbalancing:ap-south-1:966001267394:loadbalancer/app/my-alb/811106bb33a240ed",
            "Port": 80,
            "Protocol": "HTTP",
            "DefaultActions": [
                {
                    "Type": "forward",
                    "TargetGroupArn": "arn:aws:elasticloadbalancing:ap-south-1:966001267394:targetgroup/my-targets/9ab2b86bcd1a70cd",
                    "ForwardConfig": {
                        "TargetGroups": [
                            {
                                "TargetGroupArn": "arn:aws:elasticloadbalancing:ap-south-1:966001267394:targetgroup/my-targets/9ab2b86bcd1a70cd",
                                "Weight": 1
                            }
                        ],
                        "TargetGroupStickinessConfig": {
                            "Enabled": false
                        }
                    }
                }
            ]
        }
    ]
}
```

Step 12: Create Listener

STEP 13: Testing

```
suhani@debian:~$ aws elbv2 describe-load-balancers
{
    "LoadBalancers": [
        {
            "LoadBalancerArn": "arn:aws:elasticloadbalancing:ap-south-1:966001267394:loadbalancer/app/my-alb/811106bb33a240ed",
            "DNSName": "my-alb-1576251582.ap-south-1.elb.amazonaws.com",
            "CanonicalHostedZoneId": "ZP97RAFLXTNZK",
            "CreatedTime": "2026-04-30T01:06:51.261000+00:00",
            "LoadBalancerName": "my-alb",
            "Scheme": "internet-facing",
            "VpcId": "vpc-032a761867508e4ea",
            "State": {
                "Code": "active"
            },
            "Type": "application",
            "AvailabilityZones": [
                {
                    "ZoneName": "ap-south-1a",
                    "SubnetId": "subnet-001b00325d5848dca",
                    "LoadBalancerAddresses": []
                },
                {
                    "ZoneName": "ap-south-1b",
                    "SubnetId": "subnet-0088364c14b7919b0",
{
    "LoadBalancers": [
        {
            "LoadBalancerArn": "arn:aws:elasticloadbalancing:ap-south-1:966001267394:loadbalancer/app/my-alb/811106bb33a240ed",
            "DNSName": "my-alb-1576251582.ap-south-1.elb.amazonaws.com",
            "CanonicalHostedZoneId": "ZP97RAFLXTNZK",
            "CreatedTime": "2026-04-30T01:06:51.261000+00:00",
            "LoadBalancerName": "my-alb",
            "Scheme": "internet-facing",
            "VpcId": "vpc-032a761867508e4ea",
            "State": {
                "Code": "active"
            },
            "Type": "application",
            "AvailabilityZones": [
                {
                    "ZoneName": "ap-south-1a",
                    "SubnetId": "subnet-001b00325d5848dca",
                    "LoadBalancerAddresses": []
                },
                {
                    "ZoneName": "ap-south-1b",
                    "SubnetId": "subnet-0088364c14b7919b0",
                    "LoadBalancerAddresses": []
                }
            ],
            "SecurityGroups": [
                "sg-0b3fb290e628148ef"
            ],
            "IpAddressType": "ipv4"
        }
    ]
}
```